

Blocco 4 - Gentle Introduction a SQL

Sintassi, lettura delle query ed esempi progressivi

Relational Databases & SQL

Materiale aggiuntivo

Perché SQL ora

Nel blocco 3 abbiamo progettato relazioni; nel blocco 4 impariamo a leggerle con domande precise.

- ▶ nel blocco 3 una relazione aveva una granularità: “una riga per ...”;
- ▶ in SQL una tabella è il punto di partenza di una domanda;
- ▶ una query non deve solo funzionare: deve rendere esplicita la domanda;
- ▶ prima si decide la relazione giusta, poi si scrive la sintassi;
- ▶ una buona query si legge quasi come una frase controllabile.

Domanda informale

Quali prodotti attivi costano più di 100 euro, ordinati dal più caro?

Domanda SQL

Da `products`, tieni solo le righe attive con prezzo maggiore di 100, mostra alcune colonne e ordina per prezzo decrescente.

La prima query completa

```
SELECT product_id, sku, product_name, unit_price
FROM products
WHERE active = true AND unit_price > 100
ORDER BY unit_price DESC;
```

La query è breve, ma contiene già quattro decisioni: tabella sorgente, colonne, righe, ordinamento.

- ➊ FROM: da quale relazione parto?
- ➋ WHERE: quali righe tengo?
- ➌ SELECT: quali colonne mostro o calcolo?
- ➍ ORDER BY: in quale ordine leggo il risultato?
- ➎ LIMIT: quante righe finali voglio vedere?

SELECT e FROM

Partiamo dalla relazione giusta e scegliamo le colonne da mostrare.

Sintassi minima

```
SELECT colonna_1, colonna_2  
FROM tabella;
```

FROM sceglie la relazione sorgente. SELECT sceglie le colonne del risultato.

Esempio: catalogo prodotti

```
SELECT product_id, sku, product_name, category  
FROM products;
```

La granularità resta quella della tabella sorgente: una riga per prodotto.

Perché evitare SELECT *

```
SELECT *  
FROM products;
```

SELECT * è utile per esplorare, ma non è una buona risposta finale: mostra colonne non richieste e rende fragile il risultato.

Richiamo al blocco 3

- ▶ caso biblioteca: se parto da `books`, ottengo una riga per libro;
- ▶ se parto da `book_copies`, ottengo una riga per copia fisica;
- ▶ se parto da `loans`, ottengo una riga per prestito;
- ▶ la tabella scelta decide il significato di ogni riga del risultato.

Alias di colonna

```
SELECT product_name AS nome_prodotto,  
       unit_price AS prezzo_unitario  
FROM products;
```

AS assegna un nome leggibile a una colonna nel risultato.

WHERE

Il filtro decide quali righe entrano nel risultato.

Sintassi del filtro

```
SELECT colonne  
FROM tabella  
WHERE condizione;
```

La condizione viene valutata riga per riga. Se è vera, la riga passa.

Confronti semplici

```
SELECT product_id, product_name, unit_price  
FROM products  
WHERE unit_price >= 500;
```

Operatori comuni: =, <>, <, <=, >, >=.

Condizioni composte

```
SELECT product_id, product_name, category, unit_price
FROM products
WHERE category = 'hardware'
      AND unit_price >= 500;
```

AND richiede che entrambe le condizioni siano vere.

OR: alternative ammesse

```
SELECT order_id, order_date, status
FROM orders
WHERE status = 'pending'
      OR status = 'shipped';
```

OR tiene una riga se almeno una condizione è vera.

IN: elenco di valori

```
SELECT order_id, order_date, status
FROM orders
WHERE status IN ('pending', 'shipped', 'refunded');
```

IN rende più leggibili filtri con molte alternative.

BETWEEN: intervalli

```
SELECT order_id, order_date, status
FROM orders
WHERE order_date BETWEEN DATE '2025-03-01'
                  AND DATE '2025-03-31';
```

BETWEEN include gli estremi. È comodo per date e numeri.

LIKE: ricerca testuale semplice

```
SELECT product_id, product_name  
FROM products  
WHERE product_name LIKE '%Laptop%';
```

% significa “qualunque sequenza di caratteri”.

NULL

Un valore mancante non si confronta con l'uguale.

Che cos'è NULL

- ▶ NULL significa assenza di valore noto;
- ▶ non è zero;
- ▶ non è stringa vuota;
- ▶ non è uguale a un altro NULL;
- ▶ va trattato con `IS NULL` o `IS NOT NULL`.

Errore comune

```
-- Da evitare  
SELECT shipment_id, order_id  
FROM shipments  
WHERE delivered_at = NULL;
```

Questa query non trova le spedizioni non consegnate.

Forma corretta

```
SELECT shipment_id, order_id, shipped_at, delivered_at  
FROM shipments  
WHERE delivered_at IS NULL;
```

IS NULL verifica esplicitamente l'assenza del valore.

Richiamo al blocco 3

- ▶ nel caso manutenzione, `closed_at` può mancare se l'intervento è aperto;
- ▶ nel caso biblioteca, `returned_at` può mancare se il prestito è in corso;
- ▶ nel monitoraggio ambientale, una misura può avere qualità mancante o non validata;
- ▶ NULL deve sempre avere un significato di dominio.

ORDER BY e LIMIT

Il risultato di una query non ha ordine stabile se non lo dichiariamo.

Ordinamento crescente

```
SELECT product_id, product_name, unit_price  
FROM products  
ORDER BY unit_price ASC;
```

ASC ordina dal più piccolo al più grande. È il default.

Ordinamento decrescente

```
SELECT product_id, product_name, unit_price  
FROM products  
ORDER BY unit_price DESC;
```

DESC ordina dal più grande al più piccolo.

Pari merito: ordinamento stabile

```
SELECT product_id, product_name, unit_price  
FROM products  
ORDER BY unit_price DESC, product_id ASC;
```

La seconda colonna decide l'ordine quando il prezzo è uguale.

LIMIT: leggere solo le prime righe

```
SELECT product_id, product_name, unit_price  
FROM products  
ORDER BY unit_price DESC, product_id  
LIMIT 5;
```

LIMIT ha senso solo se l'ordinamento è chiaro.

Colonne calcolate

Una query può produrre valori derivati dai dati sorgente.

Espressioni aritmetiche

```
SELECT order_id, product_id,  
       quantity * unit_price AS gross_amount  
FROM order_items;
```

La colonna `gross_amount` non esiste nella tabella: viene calcolata nel risultato.

Sconto e arrotondamento

```
SELECT order_id, product_id,  
       round(quantity * unit_price *  
             (1 - discount_pct / 100.0), 2) AS net_amount  
FROM order_items;
```

L'alias rende esplicito il significato del calcolo.

CASE: classificare righe

```
SELECT product_id, product_name, unit_price,  
       CASE  
         WHEN unit_price < 100 THEN 'low'  
         WHEN unit_price < 700 THEN 'medium'  
         ELSE 'high'  
       END AS price_band  
FROM products;
```

CASE letto come regola

- ▶ CASE valuta le condizioni dall'alto verso il basso;
- ▶ la prima condizione vera produce il valore corrispondente;
- ▶ ELSE gestisce tutti i casi non catturati prima;
- ▶ senza ELSE, i casi non catturati diventano NULL;
- ▶ ogni ramo deve essere coerente con la specifica business.

Costruire una query

Scrivere SQL è un processo incrementale, non un salto diretto alla query finale.

Metodo in quattro passaggi

- ① scrivere la domanda in forma informale;
- ② scegliere la tabella con la granularità corretta;
- ③ aggiungere un filtro alla volta;
- ④ scegliere colonne, alias e ordinamento.

Esempio guidato: domanda

Domanda

Mostrare i clienti italiani registrati da febbraio 2025 in poi, con nome, città, segmento e data di registrazione, ordinati per data.

Granularità

Una riga per cliente.

Passo 1: tabella sorgente

```
SELECT *  
FROM customers;
```

Serve solo per orientarsi. Non è ancora la query finale.

Passo 2: colonne utili

```
SELECT customer_id, full_name, city, segment, signup_date  
FROM customers;
```

Ora il risultato mostra solo ciò che serve alla domanda.

Passo 3: filtri

```
SELECT customer_id, full_name, city, segment, signup_date
FROM customers
WHERE country = 'IT'
      AND signup_date >= DATE '2025-02-01';
```

Passo 4: ordinamento e alias

```
SELECT customer_id,  
       full_name AS customer_name,  
       city,  
       segment,  
       signup_date  
FROM customers  
WHERE country = 'IT'  
      AND signup_date >= DATE '2025-02-01'  
ORDER BY signup_date, customer_id;
```

Richiami ai casi del blocco 3

La sintassi resta la stessa: cambia solo la relazione di partenza.

Energia: letture anomale

```
-- Schema concettuale dal caso Energia
SELECT reading_id, month, kwh, quality
FROM energy_readings
WHERE quality <> 'valid'
ORDER BY month, reading_id;
```

La forma è la stessa: tabella, filtro, colonne, ordinamento.

Monitoraggio ambientale: misure alte

```
-- Schema concettuale dal caso Monitoraggio ambientale
SELECT measurement_id, measured_at, value, quality_flag
FROM measurements
WHERE value > 40
ORDER BY measured_at, measurement_id;
```

Non serve ancora il join: stiamo imparando a interrogare una relazione alla volta.

Esercizi brevi da discutere

- ▶ prodotti attivi sotto i 100 euro;
- ▶ clienti enterprise registrati dopo febbraio;
- ▶ ordini pending o shipped;
- ▶ spedizioni non ancora consegnate;
- ▶ righe ordine con importo netto superiore a 1000;
- ▶ prodotti classificati in fasce di prezzo.

Checklist finale

- ① Ho scelto la relazione con la granularità giusta?
- ② Ho scritto colonne esplicite invece di `SELECT *`?
- ③ I filtri corrispondono alla domanda?
- ④ Ho usato `IS NULL` per valori mancanti?
- ⑤ Le colonne calcolate hanno alias chiari?
- ⑥ L'ordinamento è dichiarato quando serve?

- ▶ SQL è un linguaggio per formulare domande precise sulle relazioni.
- ▶ FROM, WHERE, SELECT e ORDER BY hanno ruoli diversi.
- ▶ La query finale nasce per passaggi: tabella, filtro, colonne, calcoli, ordinamento.